## Q4 – Choosing a Map for a Pharmacy Inventory (6 Marks, K4)

All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

### 1. Design and predictions (before implementation)

Data: `Map<String, Medicine>` keyed by medicine code. Operations: hundreds of lookups per day; every evening two reports: (R1) all medicines in **code order**, (R2) all medicines in the **order they were added**.

| Map | put | get | Iteration | Natural order of iteration | Extra work for the wrong report |
|---|---|---|---|---|---|
| `HashMap` | O(1) average | O(1) average | O(n + capacity) | none (depends on hash values) | **both** reports need work: R1: copy keys and sort O(n log n); R2: impossible unless arrival order is stored separately |
| `TreeMap` | O(log n) | O(log n) | O(n), already sorted | **code order** (R1) | R2: needs a separate arrival list (or a sequence number and a sort) |
| `LinkedHashMap` | O(1) average | O(1) average | O(n) | **insertion order** (R2) | R1: copy keys and sort once: O(n log n) |

## 2. Implementation and measurements

```
import java.util.*;

public class MapBench {
    record Medicine(String code, String name, int stock) {}
    static double sink;
    static double ms(Runnable r) { long t = System.nanoTime();
r.run(); return (System.nanoTime() - t) / 1e6; }

    static Map<String, Medicine> build(String kind) {
        return switch (kind) { case "HashMap" -> new HashMap<>(); case
"TreeMap" -> new TreeMap<>(); case "LinkedHashMap" -> new
LinkedHashMap<>(); default -> new Hashtable<>(); };
    }
    public static void main(String[] args) {
        String[] kinds = {"HashMap", "TreeMap", "LinkedHashMap",
"Hashtable"};
        int[] sizes = {1_000, 10_000, 100_000, 500_000};
        // warm-up
        for (String k : kinds) { Map<String, Medicine> m = build(k);
for (int i = 0; i < 20000; i++) m.put("M" + i, new Medicine("M" + i,
"n", i)); for (int i = 0; i < 20000; i++) sink += m.get("M" +
i).stock(); }
        System.out.printf("%-14s %8s | %9s | %12s | %10s%n", "map",
"n", "put all", "100k lookups", "iterate");
        for (int n : sizes) {
            String[] codes = new String[n]; for (int i = 0; i < n; i+
+) codes[i] = "MED" + String.format("%07d", i);
            List<String> shuffled = new
ArrayList<>(Arrays.asList(codes)); Collections.shuffle(shuffled, new
Random(42));   // arrival order != code order
            Random rnd = new Random(7); String[] probes = new
String[100_000]; for (int i = 0; i < probes.length; i++) probes[i] =
codes[rnd.nextInt(n)];
            for (String k : kinds) {
                int reps = n >= 500_000 ? 2 : 4; double put = 0, look
= 0, it = 0;
                for (int r = 0; r < reps; r++) {
                    Map<String, Medicine> m = build(k);
                    put += ms(() -> { for (String c : shuffled)
m.put(c, new Medicine(c, "x", 10)); });
                    look += ms(() -> { double s = 0; for (String p :
probes) s += m.get(p).stock(); sink += s; });
                    it += ms(() -> { double s = 0; for (Medicine md :
m.values()) s += md.stock(); sink += s; });
                }
                System.out.printf("%-14s %8d | %9.1f | %12.1f | %10.2f
%n", k, n, put / reps, look / reps, it / reps);
            }
        }
```

```
    }
}
```

**Time (ms), keys inserted in shuffled order so that arrival order ≠ code order. Lookups: 100,000 random existing codes.**

| Map | n | put all n | 100,000 lookups | iterate all |
|---|---|---|---|---|
| HashMap | 1,000 | 0.2 | 2.0 | 0.10 |
| TreeMap | 1,000 | 0.3 | 11.7 | 0.12 |
| LinkedHash Map | 1,000 | 0.2 | 1.8 | 0.12 |
| Hashtable | 1,000 | 0.1 | 2.2 | 0.30 |
| HashMap | 10,000 | 2.0 | 1.9 | 0.45 |
| TreeMap | 10,000 | 3.8 | 29.1 | 0.51 |
| LinkedHash Map | 10,000 | 3.4 | 1.8 | 0.18 |
| Hashtable | 10,000 | 0.8 | 2.0 | 0.33 |
| HashMap | 100,000 | 21.8 | 5.9 | 4.48 |
| TreeMap | 100,000 | 110.6 | 87.2 | 8.90 |
| LinkedHash Map | 100,000 | 30.8 | 6.2 | 1.27 |
| Hashtable | 100,000 | 32.1 | 9.5 | 5.81 |
| HashMap | 500,000 | 167.3 | 13.9 | 31.66 |
| TreeMap | 500,000 | 764.5 | 159.0 | 87.54 |
| LinkedHash Map | 500,000 | 169.8 | 10.1 | 9.59 |
| Hashtable | 500,000 | 150.2 | 14.4 | 39.50 |

**Interpretation.** Hash-based maps: 100,000 lookups cost about 10–14 ms even at 500,000 medicines (≈ 0.1 µs each), almost independent of n (O(1)). `TreeMap` lookups grow with n (11.7 → 159 ms, ≈ 1.6 µs each at 500,000): O(log n) plus cache misses while descending the tree. (The 1,000-item timings include JIT warm-up noise, e.g. the high `TreeMap` figure.) `LinkedHashMap` iterates fastest at large n (9.6 vs 31.7 ms for `HashMap` at 500,000): the linked entries are visited directly, while `HashMap` has to scan the whole bucket table.

## 3. The two reports (`ReportOrder.java`)

```
import java.util.*;

public class ReportOrder {
    static double ms(Runnable r) { long t = System.nanoTime();
r.run(); return (System.nanoTime() - t) / 1e6; }
    public static void main(String[] args) {
        // small hand-checkable data set
```

```
        String[] arrival = {"PAR20", "AMX10", "IBU30", "CET05",
"DOL15"};
        Map<String, Integer> h = new HashMap<>(), t = new TreeMap<>(),
l = new LinkedHashMap<>();
        int q = 10; for (String c : arrival) { h.put(c, q); t.put(c,
q); l.put(c, q); q += 10; }
        System.out.println("arrival order        : " +
Arrays.toString(arrival));
        System.out.println("HashMap iteration    : " + h.keySet() + "
<- neither code order nor arrival order");
        System.out.println("TreeMap iteration    : " + t.keySet() + "
<- code order (report 1)");
        System.out.println("LinkedHashMap iterate: " + l.keySet() + "
<- insertion order (report 2)");

        System.out.println("\nReport timing (ms) and whether order is
right");
        System.out.printf("%8s | %-34s | %-34s%n", "n", "code-order
report", "insertion-order report");
        for (int n : new int[]{10_000, 100_000, 500_000}) {
            List<String> codes = new ArrayList<>(); for (int i = 0; i
< n; i++) codes.add("MED" + String.format("%07d", i));
            Collections.shuffle(codes, new Random(1));
            Map<String, Integer> hm = new HashMap<>(), tm = new
TreeMap<>(), lm = new LinkedHashMap<>();
            for (String c : codes) { hm.put(c, 1); tm.put(c, 1);
lm.put(c, 1); }
            double a = 0, b = 0, c = 0, d = 0, e = 0; int reps = 3;
boolean ok1 = true, ok2 = true;
            for (int r = 0; r < reps; r++) {
                a += ms(() -> { List<String> k = new
ArrayList<>(tm.keySet()); });                       // TreeMap:
already sorted
                b += ms(() -> { List<String> k = new
ArrayList<>(hm.keySet()); Collections.sort(k); });   // HashMap: extra
sort O(n log n)
                c += ms(() -> { List<String> k = new
ArrayList<>(lm.keySet()); Collections.sort(k); });   // LinkedHashMap:
extra sort
                d += ms(() -> { List<String> k = new
ArrayList<>(lm.keySet()); });                       // LinkedHashMap
insertion order
            }
            List<String> k1 = new ArrayList<>(hm.keySet());
List<String> k2 = new ArrayList<>(lm.keySet());
            ok1 = !k1.equals(codes); ok2 = k2.equals(codes);
            System.out.printf("%8d | TreeMap %.1f | HashMap+sort %.1f
| LHM+sort %.1f | insertion: LHM %.1f (HashMap in arrival order? %b,
LHM? %b)%n", n, a / reps, b / reps, c / reps, d / reps, !ok1, ok2);
        }
```

```
    }
}
```

Small check you can verify by hand (codes arrive in order PAR20, AMX10, IBU30, CET05, DOL15):

```
HashMap iteration     : [AMX10, DOL15, CET05, PAR20, IBU30]   neither
code order nor arrival order
TreeMap iteration     : [AMX10, CET05, DOL15, IBU30, PAR20]   code
order: correct for R1
LinkedHashMap iterate : [PAR20, AMX10, IBU30, CET05, DOL15]   arrival
order: correct for R2
```

| n | R1 `TreeMap` (copy keys) | R1 `HashMap` + sort | R1 `LinkedHashMap` + sort | R2 `LinkedHashMap` (copy keys) | Order correct? |
|---|---|---|---|---|---|
| 10,000 | 1.4 ms | 4.2 ms | 6.2 ms | 0.4 ms | TreeMap ✔️ R1, LinkedHashMap ✔️ R2, HashMap ✘ both |
| 100,000 | 16.0 ms | 25.4 ms | 59.0 ms | 3.1 ms | same |
| 500,000 | 30.1 ms | 103.4 ms | 281.3 ms | 12.4 ms | same |

The "wrong" map needs an extra O(n log n) sort (R1 with `HashMap`/`LinkedHashMap`: 103–281 ms at 500,000), and with a `HashMap` the arrival-order report cannot be produced at all.

## 4. Scalability inquiry

**Acceptable response time (my definition):** a pharmacist lookup must complete in **under 1 ms** in the data layer (so that the screen responds instantly, well under the ≈ 100 ms at which users notice delay), and the evening report in **under 5 seconds**. Hundreds of lookups per day is a tiny load: even the slowest structure (`TreeMap`, ≈ 1.6 µs per lookup at 500,000 medicines) meets the criterion by a factor of about 600.

**Which map I would deploy:** `LinkedHashMap<String, Medicine>`. Lookups are the fastest and constant time (≈ 0.1 µs); R2 comes free (iteration order = insertion order); R1 needs one sort of the keys per evening, which at 500,000 medicines costs about 281 ms (a real hospital pharmacy has a few thousand, i.e. a few milliseconds) — well inside the 5-second budget. If the code-ordered view were needed continuously (for example a screen showing medicines between two codes) I would use a `TreeMap` as the main map plus an `ArrayList` of codes for insertion order, since it is the only one that supports range queries.

**Where the remaining cost comes from:** (1) building the map, which allocates one entry object per medicine plus the `Medicine` objects (put all: 170 ms for 500,000); (2) hashing and comparing `String` keys; (3) memory access: at large n the table and objects do not fit in the CPU cache, so each lookup incurs cache misses, which is why even O(1) lookup times rose with n (2.0 → 13.9 ms); (4) for `TreeMap`, the O(log n) tree descent with pointer chasing.

**Why `Hashtable` is not preferred in new code:** it is a legacy class with every method `synchronized`, so it pays for locking even single-threaded and still is not safe for compound operations (check-then-put); it does not allow `null` keys or values; and it has no ordered variant. My results show no performance advantage (14.4 vs 13.9 ms for lookups at 500,000, slower iteration, similar `put`). Use `HashMap` for single-threaded code and `ConcurrentHashMap` when several threads share the map.

## 5. Alternative concept and complexity analysis

**A sorted array of medicines searched with `Arrays.binarySearch()`** (`SortedArrayInventory.java`):

```
import java.util.*;

/** Alternative: array of medicine codes sorted, searched with
Arrays.binarySearch. */
public class SortedArrayInventory {
    public static void main(String[] args) {
        for (int n : new int[]{10_000, 100_000, 500_000}) {
            String[] codes = new String[n]; for (int i = 0; i < n; i+
+) codes[i] = "MED" + String.format("%07d", i);
            Random rnd = new Random(7); String[] probes = new
String[100_000]; for (int i = 0; i < probes.length; i++) probes[i] =
codes[rnd.nextInt(n)];
            for (int w = 0; w < 2; w++) for (String p : probes)
Arrays.binarySearch(codes, p);
            long t0 = System.nanoTime(); long s = 0; for (String p :
probes) s += Arrays.binarySearch(codes, p); double look =
(System.nanoTime() - t0) / 1e6;
            // insertion of 1000 new codes keeping order: binary
search + shift = O(n) each
            String[] arr = Arrays.copyOf(codes, n + 1000); int size =
n; Random r2 = new Random(3);
            t0 = System.nanoTime();
            for (int i = 0; i < 1000; i++) {
                String c = "MED" + String.format("%07d",
r2.nextInt(n)) + "x";
                int pos = Arrays.binarySearch(arr, 0, size, c); if
(pos < 0) pos = -pos - 1;
                System.arraycopy(arr, pos, arr, pos + 1, size - pos);
arr[pos] = c; size++;
            }
            double ins = (System.nanoTime() - t0) / 1e6;
            System.out.printf("n=%7d  100k binarySearch lookups = %.1f
```

```
ms   1000 ordered inserts (arraycopy shift) = %.1f ms   code-order
report = O(n) direct scan (already sorted)%n", n, look, ins);
        }
    }
}
```

| n | 100,000 binary-search lookups | 1,000 ordered inserts (`binarySearch` + `System.arraycopy`) |
|---|---|---|
| 10,000 | 33.2 ms | 15.1 ms* |
| 100,000 | 69.4 ms | 8.1 ms* |
| 500,000 | 93.7 ms | 24.5 ms |

* The 10,000 figure is larger than the 100,000 one, which shows measurement noise (JIT and garbage collection) at these tiny durations; repeat to confirm the trend (insert time should grow with n).

| | Sorted array + binarySearch | `HashMap` | `TreeMap` |
|---|---|---|---|
| Lookup | O(log n) | O(1) average | O(log n) |
| Insert | **O(n)** (find position in O(log n), then shift n/2 elements on average) | O(1) average | O(log n) |
| Code-order report | **O(n)**, directly (already sorted; no sort needed) | O(n log n) (sort keys) | O(n) |
| Insertion-order report | not available without a separate list | not available | not available |
| Space | **smallest**: one reference per medicine (≈ 4–8 bytes), no entry objects | table + one entry object per item (≈ 32+ bytes) | one tree node per item (≈ 40 bytes) |

The sorted array is a good choice when the inventory is **mostly read and rarely changed** (it gives the code-ordered report for free and uses the least memory), but each insertion costs O(n) (shifting), so it is not suited to inventory that changes all day. Lookup speed was between the hash maps (13.9 ms) and `TreeMap` (159 ms) in my tests (93.7 ms at 500,000): binary search on a contiguous array is cache-friendlier than a tree.